

Project Work - Museo Civico Aurora

DESCRIZIONE DELLA PROVA

Contesto

Il museo cittadino “Museo Civico Aurora” gestisce mostre temporanee, opere permanenti, visite guidate e vendita di biglietti per i visitatori.

Attualmente la gestione delle informazioni è frammentata tra diversi strumenti: fogli di calcolo, documenti condivisi, comunicazioni via email e aggiornamenti manuali del sito web. Questa situazione rende complesso mantenere aggiornate le informazioni sulle mostre, pubblicare contenuti relativi alle opere, controllare le disponibilità delle visite guidate e gestire in modo ordinato le prenotazioni dei visitatori.

Il museo desidera quindi realizzare una nuova piattaforma digitale che permetta di centralizzare la gestione dei contenuti e dei servizi offerti al pubblico.

La piattaforma dovrà consentire al personale del museo di inserire e aggiornare mostre, opere, immagini e visite guidate. I visitatori dovranno invece poter consultare le informazioni pubblicate, visualizzare le mostre disponibili, consultare le opere esposte, prenotare una visita guidata e acquistare o registrare un biglietto.

Il progetto rappresenta un primo Proof of Concept della soluzione. Se il sistema risulterà funzionale, potrà essere esteso in futuro con ulteriori funzionalità, come autenticazione degli utenti, pagamenti online reali, gestione avanzata degli accessi, notifiche email e statistiche sulle visite.

OBIETTIVI DELL'APPLICAZIONE

L'obiettivo è sviluppare un'applicazione full-stack per la gestione digitale di un museo.

Il sistema dovrà permettere:

- la pubblicazione di mostre, opere e contenuti multimediali;
- la consultazione pubblica delle informazioni da parte dei visitatori;
- la gestione delle visite guidate disponibili;
- la prenotazione delle visite guidate;
- la registrazione dell'acquisto dei biglietti;
- la gestione amministrativa dei contenuti da parte del personale del museo.

Il risultato atteso è una soluzione software funzionante, composta da backend, frontend e database, in grado di gestire un flusso dati completo dalla creazione dei contenuti fino alla loro visualizzazione e fruizione da parte degli utenti.

REQUISITI FUNZIONALI

Registrazione e autenticazione degli utenti

La registrazione e l'autenticazione degli utenti non sono obbligatorie in questa versione.

È possibile simulare il ruolo del visitatore e del personale del museo attraverso pagine o funzionalità distinte.

L'autenticazione potrà essere considerata come possibile estensione futura del progetto.

Gestione delle mostre

L'applicazione deve permettere la gestione delle mostre presenti nel museo.

Per ogni mostra dovranno essere previste almeno le seguenti informazioni:

- titolo;
- descrizione;
- data di inizio;
- data di fine;
- immagine principale;
- stato della mostra, ad esempio attiva, programmata o conclusa.

Il personale del museo dovrà poter creare, modificare, eliminare e visualizzare le mostre. I visitatori dovranno poter visualizzare l'elenco delle mostre disponibili e consultare il dettaglio di una singola mostra.

Gestione delle opere

L'applicazione deve permettere la gestione delle opere presenti nel museo.

Per ogni opera dovranno essere previste almeno le seguenti informazioni:

- titolo;
- autore;
- anno di realizzazione;
- descrizione;
- tecnica o tipologia dell'opera;
- immagine dell'opera;
- mostra di appartenenza, se presente.

Il personale del museo dovrà poter creare, modificare, eliminare e visualizzare le opere. I visitatori dovranno poter consultare l'elenco delle opere e visualizzare il dettaglio di una singola opera.

Pubblicazione di contenuti testuali e immagini

L'applicazione deve permettere la pubblicazione di contenuti descrittivi relativi alle mostre e alle opere.

Le immagini possono essere gestite tramite URL, senza la necessità di implementare un sistema di upload fisico dei file.

È sufficiente memorizzare nel database il percorso o l'indirizzo dell'immagine. Esempio:
<https://example.com/images/opera1.jpg>

Non è obbligatorio implementare la gestione avanzata dei file multimediali.

Gestione delle visite guidate

L'applicazione deve permettere la gestione delle visite guidate organizzate dal museo.

Per ogni visita guidata dovranno essere previste almeno le seguenti informazioni:

- titolo della visita;
- descrizione;
- data e ora di svolgimento;
- durata prevista;
- nome della guida;
- numero massimo di partecipanti;
- mostra associata, se presente.

Il personale del museo dovrà poter creare, modificare, eliminare e visualizzare le visite guidate. I visitatori dovranno poter visualizzare l'elenco delle visite guidate disponibili.

Prenotazione delle visite guidate

L'applicazione deve permettere ai visitatori di prenotare una visita guidata.

Per ogni prenotazione dovranno essere previste almeno le seguenti informazioni:

- nome del visitatore;
- cognome del visitatore;
- email;
- numero di partecipanti;
- visita guidata selezionata;
- data della prenotazione;
- stato della prenotazione, ad esempio confermata o annullata.

Il sistema dovrà impedire, o almeno segnalare, il superamento della capienza massima prevista per una visita guidata.

Acquisto o registrazione dei biglietti

L'applicazione deve permettere la registrazione dell'acquisto di biglietti.

Per ogni biglietto o acquisto dovranno essere previste almeno le seguenti informazioni:

- nome del visitatore;
- cognome del visitatore;
- email;
- tipologia di biglietto, ad esempio intero, ridotto, gratuito;
- quantità;
- prezzo totale;
- data di acquisto;
- mostra o visita guidata associata, se presente.

Non è richiesta l'integrazione con sistemi di pagamento reali. Il pagamento può essere simulato registrando semplicemente l'acquisto nel database.

REQUISITI NON FUNZIONALI

Usabilità

L'applicazione web deve essere semplice da utilizzare e deve permettere una navigazione chiara tra le diverse sezioni: mostre, opere, visite guidate, prenotazioni, biglietti e area di gestione per il personale del museo.

Persistenza dei dati

La soluzione dovrà utilizzare PostgreSQL/MySQL/SQL Server per la persistenza dei dati. Il candidato dovrà progettare le tabelle necessarie e definire correttamente le relazioni tra le entità principali.

API REST

Il backend dovrà esporre API REST sviluppate con C# / ASP.NET Core. Le API dovranno permettere almeno: lettura degli elenchi, lettura del dettaglio, inserimento di nuovi dati, modifica dei dati esistenti ed eliminazione dei dati, dove previsto.

Frontend Web

L'applicazione web dovrà essere sviluppata utilizzando Blazor. Il frontend dovrà comunicare con il backend per visualizzare, inserire e modificare i dati.

Gestione degli errori

L'applicazione dovrà gestire correttamente le principali situazioni di errore: dati obbligatori mancanti, risorsa non trovata, prenotazione con numero di partecipanti superiore alla disponibilità, errore durante il salvataggio dei dati e formato non valido dei dati inseriti dall'utente.

COMPITI

La prova si struttura in quattro compiti principali.

COMPITO 1 – PROGETTAZIONE ARCHITETTURALE

Descrivere il sistema a livello di contesto, architettura e flussi di dati, anche attraverso l'utilizzo di opportuni diagrammi.

La descrizione deve includere:

1. Contesto e attori coinvolti

Dettagliare il contesto operativo del “Museo Civico Aurora” e gli attori coinvolti.

Gli attori principali sono:

- visitatore;
- personale del museo;
- sistema backend;
- applicazione web;
- database PostgreSQL/MySQL/SQL Server.

Si consiglia l'utilizzo di diagrammi dei casi d'uso.

2. Componenti software previste

Elencare e descrivere le componenti software principali del sistema. A titolo di esempio:

- applicazione frontend Blazor;
- API REST ASP.NET Core;
- database PostgreSQL/MySQL/SQL Server;
- eventuale servizio o classe per la gestione delle prenotazioni;
- eventuale servizio o classe per la gestione dei biglietti;
- eventuale servizio o classe per la gestione dei contenuti.

3. Flussi di dati tra le componenti

Descrivere i flussi di dati tra le varie componenti del sistema. Esempi di flussi da descrivere:

- il personale del museo crea una nuova mostra;
- il visitatore visualizza l'elenco delle mostre;
- il visitatore consulta il dettaglio di un'opera;
- il visitatore prenota una visita guidata;

- il visitatore acquista o registra un biglietto.

Si consiglia l'utilizzo di diagrammi di sequenza o diagrammi di flusso per rappresentare il passaggio dei dati tra frontend, backend e database.

COMPITO 2 – PROGETTAZIONE MODELLI DATI E STRUTTURA DEL DATABASE

Progettare i modelli dati necessari per rappresentare le informazioni relative a mostre, opere, visite guidate, prenotazioni e biglietti.

Il candidato dovrà definire:

- le tabelle necessarie;
- le chiavi primarie;
- le chiavi esterne;
- le relazioni tra le tabelle;
- i tipi di dato;
- eventuali vincoli di obbligatorietà;
- eventuali valori di default.

Il database dovrà essere realizzato utilizzando PostgreSQL/MySQL/SQL Server.

Entità minime consigliate:

- Exhibitions, per la gestione delle mostre;
- **Artworks, per la gestione delle opere;**
- GuidedTours, per la gestione delle visite guidate;
- Reservations, per la prenotazione delle visite guidate;
- Tickets, per la registrazione dei biglietti.

È possibile aggiungere ulteriori tabelle se ritenute utili, ad esempio Visitors, TicketTypes, MuseumStaff, Images, Categories.

COMPITO 3 – IMPLEMENTAZIONE API E WEB APP

Implementare le API e l'applicazione web necessarie alla gestione del sistema.

Implementazione API REST

Le API dovranno includere endpoint per gestire almeno le seguenti funzionalità:

Mostre

- creare una nuova mostra;
- visualizzare l'elenco delle mostre;
- visualizzare il dettaglio di una mostra;
- modificare una mostra;
- eliminare una mostra.

Opere

- creare una nuova opera;
- visualizzare l'elenco delle opere;
- visualizzare il dettaglio di un'opera;
- modificare un'opera;
- eliminare un'opera.

Visite guidate

- creare una nuova visita guidata;
- visualizzare l'elenco delle visite guidate;
- visualizzare il dettaglio di una visita guidata;
- modificare una visita guidata;
- eliminare una visita guidata.

Prenotazioni

- creare una nuova prenotazione;
- visualizzare l'elenco delle prenotazioni;
- visualizzare le prenotazioni associate a una visita guidata;
- annullare una prenotazione.

Biglietti

- registrare un nuovo acquisto di biglietti;
- visualizzare l'elenco dei biglietti acquistati;
- visualizzare il dettaglio di un acquisto.

Le API devono essere progettate seguendo i principi RESTful. È consigliato l'utilizzo di Swagger o strumenti simili per testare gli endpoint realizzati.

Sviluppo dell'applicazione web

Creare un'applicazione web utilizzata dai visitatori e dal personale del museo.

Area visitatore

Il visitatore deve poter:

- visualizzare l'elenco delle mostre;
- consultare il dettaglio di una mostra;

- visualizzare le opere associate a una mostra;
- consultare il dettaglio di un'opera;
- visualizzare le visite guidate disponibili;
- prenotare una visita guidata;
- acquistare o registrare un biglietto.

Area personale museo

Il personale del museo deve poter:

- creare e modificare mostre;
- creare e modificare opere;
- creare e modificare visite guidate;
- visualizzare le prenotazioni ricevute;
- visualizzare i biglietti acquistati.

Non è obbligatorio implementare un sistema di autenticazione. Le due aree possono essere simulate attraverso pagine o menu separati.

Modalità di sviluppo dell'applicazione

1. Backend API + Frontend Blazor

Soluzione completa. Prevede la creazione di un backend ASP.NET Core che espone API REST, un frontend Blazor che consuma le API e un database PostgreSQL/MySQL/SQL Server. Questa è la modalità preferita.

2. Applicazione unica con rendering server-side

Soluzione intermedia. Prevede la creazione di un'unica applicazione in cui il server gestisce sia le pagine web sia la logica applicativa. La soluzione deve comunque prevedere una chiara separazione tra logica di accesso ai dati, logica applicativa e interfaccia utente.

COMPITO 4 – CONFIGURAZIONE, TEST E CONSEGNA DEL PROGETTO

Il candidato dovrà predisporre il progetto in modo che sia facilmente avviabile e verificabile.

Configurazione dell'ambiente

Il progetto dovrà includere:

- istruzioni per la creazione del database PostgreSQL/MySQL/SQL Server;
- script SQL per la creazione delle tabelle;
- eventuali script per l'inserimento di dati di esempio;

- configurazione della stringa di connessione;
- indicazioni per l'avvio del backend;
- indicazioni per l'avvio del frontend.

Test dell'applicazione

Il candidato dovrà verificare il corretto funzionamento delle principali funzionalità:

- creazione di una mostra;
- visualizzazione delle mostre;
- creazione di un'opera;
- visualizzazione delle opere;
- creazione di una visita guidata;
- prenotazione di una visita guidata;
- registrazione dell'acquisto di un biglietto.

È possibile utilizzare Swagger, Postman o l'interfaccia web realizzata.

Consegna finale

La consegna dovrà includere:

- codice sorgente completo;
- script SQL;
- eventuali dati di esempio;
- eventuale documentazione;
- eventuali diagrammi prodotti;
- istruzioni minime per l'avvio del progetto.

Il progetto dovrà essere consegnato in formato .ZIP secondo le modalità indicate nella sezione "Modalità di consegna dell'elaborato".